

```
• X_train = X_train.reshape(X_train.shape[0], 1, 28, 28).
X_test = X_test.reshape(X_test.shape[0], 1, 28, 28)
```

Convert the data to a float type and normalize the values between the range [0, 1]. Then convert the dimensionality of the entire data to another format using the code below:-

```
• X_train = X_train.astype('float32'), X_test /= 255
• Y_test = np_utils.to_categorical(y_test, 10)
```

Defining the model architecture

Defining the model architecture is important to let the system know of the neural network schema to be created or the number of hidden layers to be added. Add the input layer information and activate the function:-

```
• model = Sequential()
• model.add(Convolution2D(32, 3, 3, activation='relu',
input_shape=(1,28,28)))
• print model.output_shape model.add(Convolution2D(32, 3,
3, activation='relu'))
• model.add(MaxPooling2D(pool_size=(2,2))) model.
add(Dropout(0.25))
• model.add(Flatten()) model.add(Dense(128,
activation='relu'))
• model.add(Dropout(0.5)) model.add(Dense(10,
activation='softmax'))
```

Density of the layers plays an important part for making the models more defined and helps them in reaching the outputs faster, given the size of the layers is properly designated. Keras provides its users an edge over the others as it automatically handles the connections between layers without needing to code for each segment.

```
• model = Sequential(), model.add(Convolution2D(32, 3,
3, activation='relu', input_shape=(1,28,28))), model.
add(Convolution2D(32, 3, 3, activation='relu'))
• model.add(MaxPooling2D(pool_size=(2,2))). model.
add(Dropout(0.25))
• model.add(Flatten()), model.add(Dense(128,
activation='relu'))
• model.add(Dropout(0.5)), model.add(Dense(10,
activation='softmax'))
```

The above code prepares the environment to include the loss function